第 5-1 讲 • 那些 verify 脚本在防什么

> **模块 5 • 质量与安全** | 建议用时 **40 分钟**（阅读 20 + 动手 20）
> 前置要求：完成模块 4（你已经改过代码，也踩过坑）

本节目标

读完并做完这一讲，你能：

1. 说清**自动化验证**和"手动点一遍"的本质区别（不只是"快"）。
2. 看懂 `selftest` 那 **88 项检查**覆盖了什么，以及**为什么是这些**。
3. 掌握**回归测试**的概念：为什么改 A 功能要跑 B 功能的检查。
4. 学会**按改动范围选择该跑哪些检查**（不是每次都跑全套）。
5. 知道**什么时候该写一个新的检查**（这才是从"会用"到"会建"）。

一、为什么需要自动化验证

1.1 现实比喻：飞机的检查单

飞行员起飞前有一份检查单，几十项，逐项确认。

为什么不靠"我觉得没问题"：

问题	说明
人会漏	第 37 项在情绪紧张时很容易跳过
人会累	飞第 100 次时，注意力远不如第 1 次
人会自我说服	"上次都没事，这次不用看了"
记忆不可靠	想不起来"上次是不是也这样"

检查单解决的正是这四件事：它**不依赖人的状态**，每次都一样。

软件验证是完全一样的逻辑：

```
手动测试：打开软件 → 点一遍 → "看起来没问题" → 上线
自动验证：跑命令 → 88 项逐条确认 → 全绿才继续
```

1.2 自动化验证的三个真实优势

① 它记得住全部

人能同时记住的检查点大概 5–7 个。**88 项？不可能。** 而脚本一项都不会漏。

② 它不会"这次懒得看"

跑一次 8 秒。**人不可能每次都检查 88 项，但脚本可以。**

③ 它能防止"改 A 坏 B"

这才是最重要的。举个真实例子：

> 有人改"删除记录"的逻辑，加了"顺便删掉收据图片"。
> 结果**回收站的"恢复"功能坏了**——因为图片已经没了。

这种错误，"改完点一下删除"是发现不了的（删除看起来正常）。**只有跑一遍回收站相关的检查才会暴露。**

术语：这种"新改动破坏了旧功能"叫**回归（Regression）**，对应的检查叫**回归测试。**

二、看一遍真实的 88 项

2.1 它们检查什么（真实清单节选）

```
cd "C:\Users\34123\Desktop\记账簿"
Select-String -Path src\main\selftest.ts -Pattern "check\('([^']+)'" | ForEach-Object { $_
```

这是我从代码里抽出来的真实检查项名字：

分类	检查项（节选）
初始数据	预设账户、预设分类、二级分类
备注配色	相同备注同色、不同备注不同色
增删改查	新增支出、列表命中 3 笔、修改金额、软删除、恢复删除
金额口径	支出合计 28.50、收入合计 12000.00、**转账不计入收支**
余额计算	支付宝余额 500.00、自检甲余额 220.00
搜索	按备注搜索
附件策略	附件复制进数据目录、**软删除不动物理附件**、**彻底删除默认保留附件文件**
账户约束	**有记录账户不许真删**、**真删被拦下并给出决策**
归档	归档成功、归档账户可恢复
迁移	迁移成功、迁移了 3 笔记录、**合并了 1 笔内部转账**、源账户已归档
备份	手动备份
数据清理	清理测试数据、账户回到 0

注意那些"有态度"的检查项：

- **「转账不计入收支」** —— 这是**业务口径**的守护者。有人不小心把转账算进支出，这条立刻失败。
- **「有记录账户不许真删」** —— 防止**误删导致数据丢失**（2-3 讲的软删除设计）。
- **「彻底删除默认保留附件文件」** —— 3-4 讲我们见过界面上的默认行为，这里检查它在数据层也被遵守。

这些不是"顺手写的"，每一类都对应一个真实的风险。

2.2 为什么是 88 项这个数量

这个数字不是设计出来的，是长出来的。

阶段	大概的检查数
刚做记账功能	十几项
加了账户归档	+10
加了软删除与回收站	+10
加了附件	+8
加了迁移	+15
加了截图识别	+若干
……	累计 88

规律：**每加一个功能，就给它补几条检查。** 这样代码越长，安全网越密。

这引出一个重要的实践：

> **写新功能时，顺手给它写 2–3 条检查。**

为什么：因为**你自己写的功能，半年后你自己也会改坏。**检查是留给未来的你的。

三、按改动范围选检查（实用技能）

不是每次都该跑全套——全套 2–3 分钟，改个文字不值得。**要按风险选。**

3.1 对照表

你改了什么	该跑什么	耗时
只改文案 / 颜色 / 样式	`typecheck`	3 秒
改界面组件（表格、表单）	`typecheck` + `smoke:pages`	~1 分钟
改动效（CSS / 动画）	`verify:motion`	~1 分钟
改主进程逻辑（IPC、服务）	`typecheck` + `selftest`	~15 秒
改数据库 / 仓储层 / 迁移	`typecheck` + `selftest` + `verify:upgrade`	~30 秒
改截图识别	`verify:billshot`	几秒
改导入导出	`verify:exchange`	几秒
改应用锁 / 备份	`verify:security`	几秒
准备发布	**全套**（见下）	2–3 分钟

3.2 "全套"是什么

```
npm run typecheck     # 类型安全
npm run selftest      # 数据层逻辑（88 项）
npm run smoke:pages   # 每个页面能渲染（18 项）
npm run verify:motion # 动效没被破坏（34 项）
```

为什么是这四个：它们覆盖**四类最容易被改坏的东西**——类型、数据、页面、动效。

发布前还要加：

```
npm run verify:upgrade    # 老用户能否安全升级
npm run verify:packaged   # 打包出来的 exe 真能跑吗
```

3.3 一条实用建议

如果你不确定该跑什么，就跑全套。 2–3 分钟换"确定没坏"，非常值。

但有一条铁律：

> **改了 `db\` 目录下的任何东西（迁移、仓储层）→ 必须跑 `selftest` 和 `verify:upgrade`。**

为什么：那里是**唯一会动用户已有数据**的地方（2-3 讲说过）。**改错了，用户的账就毁了。**

四、什么时候该写新检查

4.1 三个信号

信号	说明	该写什么
发现了一个 bug	修完之后	**立刻写一条检查**，防止它下次再回来
实现了一个复杂规则	比如"转账不计入收支"	写一条断言，把规则钉住
改了一处后，你不确定有没有影响别的	心里没底	说明缺一条检查

第二条最容易被忽略：**业务规则不写进检查，就会在半年后的某次改动里被悄悄破坏**，而且**没人发现**（因为没有报错，只是数字算错了）。

这就是为什么 `selftest` 里有「转账不计入收支」这一条——它把一条重要业务口径**钉死**了。

4.2 一条检查长什么样

格式很简单（项目里用的是同一个模式）：

```
check('转账不计入收支', expenseTotal === 2850 && transferTotal === 5000, `${expenseTotal} /
```

三个参数：

参数	意思
`'转账不计入收支'`	检查项名字（**失败时会显示，所以要写清楚**）
判断条件	返回 `true` 表示通过
详情字符串	失败时显示"实际是什么"，便于排查

好的检查项名字的特点：**读起来像一句断言**（"软删除后不出现"），而不是"测试用例 12"。

为什么：因为失败时你要**一眼看出**是哪个规则被破坏了。

五、动手任务

步骤 1：看一遍全部检查项名字

```
cd "C:\Users\34123\Desktop\记账簿"
Select-String -Path src\main\selftest.ts -Pattern "check\('([^']+)'" | ForEach-Object { $_
```

扫一遍这 88 个名字，回答：

- 有没有你**没想到**的检查项？
- 有没有哪个名字你**看不懂它在检查什么**？

看不懂的那些，就是你可以去学的地方（它们通常在检查一个你可能还不知道的业务规则）。

步骤 2：验证"回归"的真实存在

任务：故意破坏一个功能，看检查能不能抓到。

① 备份：

```
Copy-Item src\main\db\repositories\records.repo.ts "$env:TEMP\repo-test.ts" -Force
```

② 破坏它：用 VS Code 打开 `src\main\db\repositories\records.repo.ts`，找到第 **85 行**：

```
else if (!query.includeDeleted) conds.push('r.deleted_at IS NULL')
```

把它改成（`IS NULL` → `IS NOT NULL`，**故意改成相反的意思**）：

```
else if (!query.includeDeleted) conds.push('r.deleted_at IS NOT NULL')
```

这一行的含义：正常查询只要"没被删除的记录"。你改成 `IS NOT NULL` 之后，**列表只会显示被删除的记录**——完全反了。

③ 跑检查：

```
npm run selftest 2>&1 | Select-String "FAIL" | Select-Object -First 6
```

我实测跑过这个实验，结果是——9 项检查同时失败：

```
FAIL  列表命中 3 笔  -  0
FAIL  支出合计 28.50  -  0
FAIL  收入合计 12000.00  -  0
FAIL  转账不计入收支  -  0
FAIL  按备注搜索  -  0
FAIL  软删除后不出现  -  ok
...
===== 数据层自检：79/88 通过 =====
```

注意「软删除后不出现」这一条失败了 —— 它的名字直接说出了你破坏的功能。

这就是"回归测试抓到了你的破坏"。 而且注意：**你只改了一行，却触发了 9 项失败**——因为这一行是所有列表查询的基础。

这说明两件事：

1. **检查网是密到能兜住这种破坏的**
2. **一处小改动的影响面，可能比你想象的大得多**（这就是为什么要有检查）

④ 还原：

```
Copy-Item "$env:TEMP\repo-test.ts" src\main\db\repositories\records.repo.ts -Force
Remove-Item "$env:TEMP\repo-test.ts" -Force
npm run selftest 2>&1 | Select-String "通过" | Select-Object -Last 1
```

预期：回到 `88/88 通过`。

这个实验让你亲眼看到：**检查不是摆设，它真的会抓到破坏。** 而且**还原之后一切都回来了**——这就是"有安全网"的感觉。

步骤 3：体验"按范围选检查"

任务：假设你要改记录列表的显示（`RecordTable.tsx`），该跑哪些？

推理：

```
改的是界面组件 → 不会碰数据 → 不用跑 selftest
但要确认页面还能渲染 → 跑 smoke:pages
类型也要确认 → 跑 typecheck
```

执行：

```
npm run typecheck
npm run smoke:pages 2>&1 | Select-String "页面冒烟"
```

预期：0 错误 + `18/18 干净`。

对比一下时间：这两个加起来约 1 分钟。而全套要 2–3 分钟。**按范围选，效率翻倍。**

步骤 4：读一条检查的实现

```
Select-String -Path src\main\selftest.ts -Pattern "转账不计入收支" -Context 3,1
```

看它怎么写的：抓取数据、算合计、断言三者的关系。

然后问自己：如果想加一条「删除记录后，账户余额应该减少」的检查，你会怎么写？

答案思路：

① 建一个账户，记一笔支出
② 记下余额
③ 删除那笔记录
④ 断言余额恢复

这就是写检查的通用套路：制造场景 → 记录状态 → 执行操作 → 断言结果。

卡住了怎么办

现象	原因	怎么办
改了但检查还全过	你的改动可能没被覆盖	**这正说明该补一条检查**
`selftest` 报错找不到模块	没构建	先 `npm run build`（或直接 `npm run selftest`，它自带构建）
输出太长找不到失败项	正常	用 `Select-String "FAIL"` 筛
实验后项目不干净	忘了还原	用备份还原，再跑 `typecheck` 确认

六、自测题

1. 自动化验证和"手动点一遍"相比，最关键的优势是什么？

2. 什么是"回归"？为什么"改完点一下"发现不了它？

3. `selftest` 里的「转账不计入收支」这条检查，防的是什么？

4. 你改了数据库迁移文件，必须跑哪两个检查？为什么？

5. 什么时候该写一条新的检查？

答案与解析

1. 它能防止"改 A 坏 B"（回归）——人手动测试时，只会测自己刚改的功能，**不会去测所有其它功能。** → 另外两个优势：不会漏（人能记住 5–7 项，脚本能记 88 项）、不依赖状态（人累了会跳过）。

2. 回归 = 新改动把原有功能弄坏了。 → 因为**被破坏的功能往往不在你测试的路径上。**比如改"删除记录"时顺手删了图片，**删除看起来一切正常**，但回收站的"恢复"坏了——你不去点恢复就发现不了。

3. 防的是**业务口径被悄悄改变**：有人不小心把转账算进支出，会导致所有统计数字错误。 → 这类错误**不会报错**（程序照常运行），只是**数字算错了**——所以必须靠断言钉住。

4. `selftest` **和** `verify:upgrade`**。** → 因为 `db\` 目录（迁移、仓储层）是**唯一会动用户已有数据**的地方。 `selftest` 验数据逻辑， `verify:upgrade` 专门验"老用户的库能不能安全升级"。**改错了，用户的账就毁了。**

5. ① **修完一个 bug 时**（防止它回来）；② **实现了一个复杂业务规则时**（把规则钉住）；③ **你心里没底"会不会影响别的"时**（说明缺检查）。 → 最简单也最有效的时机是**第 ① 个**：bug 刚修完，你脑子里最清楚它怎么坏的。

七、术语表

术语	一句话解释
自动化验证	用脚本代替人工逐项检查
回归 / Regression	新改动破坏了原有功能
回归测试	用来发现回归的检查
断言 / Assert	一条"必须成立"的判断
检查项名字	失败时显示的说明，应写得像一句断言
覆盖 / Coverage	检查覆盖了多少功能
基线	改动前的干净状态
冒烟测试 / Smoke Test	最快速的"能不能跑起来"检查
验收 vs 回归	验收验"新功能对不对"，回归验"旧功能还在不在"

八、本讲与项目的对应

- 你查看的文件：`src\main\selftest.ts`（88 项检查的实现）
- 你做的实验：故意改坏 `deleted_at` 条件 → 看检查抓到 → 还原
- **本讲不修改功能代码（实验会还原），不影响你的记账数据**
- 上一讲：4-4 读懂报错 ｜ 下一讲：**5-2 数据安全：备份、迁移、加密**

讲义版本：v1 ｜ 对应项目版本：0.1.0